

COMPARACIÓN

[HOME](#)[SERVICE](#)[ABOUT US](#)[CONTACT US](#)

Características

Sessions

Cookies

JTW (JSON Web Tokens)

Definición

Mecanismo que almacena información del usuario en el servidor de forma temporal mientras navega por una aplicación.

Pequeños archivos de texto que los sitios web envían al navegador del cliente para guardar información y rastrear su actividad.

Estándar abierto (RFC 7519) que define un formato compacto y autónomo para transmitir datos seguros entre partes como un objeto JSON.

¿Dónde se almacena la información?

En el servidor (en memoria, base de datos o sistema de archivos).

En el cliente (navegador web del usuario).

En el cliente (usualmente en LocalStorage, SessionStorage o en una Cookie).

¿Quién administra la información?

El servidor.

Principalmente el cliente

El cliente la almacena, pero el servidor genera y valida criptográficamente la información.

Nivel de seguridad

Alto. Los datos reales no se exponen al cliente; solo viaja un identificador

Medio/Bajo. Depende de la configuración; si no se aseguran, los datos están expuestos a manipulación o robo.

Alto. La información está firmada criptográficamente (previene manipulación), aunque el payload puede ser leído por cualquiera si no se encripta.



Persistencia de la información

Temporal. Generalmente desaparece al cerrar el navegador o tras un periodo corto de inactividad.

Configurable. Pueden ser temporales (de sesión) o tener una fecha de caducidad a largo plazo.

Configurable mediante el atributo de expiración (exp) dentro del propio token.



Ventajas

- Datos ocultos al usuario final.
- Fácil invalidar la sesión desde el backend.
- Soporta grandes cantidades de datos.

- Fáciles de implementar.
- Descargan de trabajo y almacenamiento al servidor.
- Mantienen el estado incluso tras cerrar el navegador.

- Stateless (no requieren almacenar el estado en el servidor).
- Altamente escalables.
- Ideales para arquitecturas de microservicios y APIs.

Desventajas

- Consumen memoria/recursos en el servidor.
- Dificultan la escalabilidad horizontal (requieren sticky sessions o una base de datos central).

- Límite estricto de tamaño (aprox. 4KB).
- Consumen ancho de banda al viajar en cada petición HTTP.

- Difíciles de invalidar/revocar manualmente antes de que expiren.
- El token aumenta de tamaño si se le agregan muchos datos.



Casos de uso

Aplicaciones web monolíticas tradicionales, portales donde se maneja información muy sensible exclusivamente en el backend.

Guardar preferencias del usuario (tema, idioma), analíticas web, rastreo de carritos de compras no autenticados.

Autenticación en APIs RESTful, Single Page Applications (SPAs), aplicaciones móviles, y sistemas Single Sign-On (SSO).

Ejemplo de aplicaciones

Foros web tradicionales, sistemas bancarios o de administración internos antiguos.

Tiendas en línea (para sugerir productos), sitios de noticias, portales de marketing.

Servicios de autenticación moderna (Google, Facebook), backends hechos en Node.js o PHP conectados a un frontend en React/Angular.

Riesgos de seguridad más comunes

- Session Hijacking
- Session Fixation

- Cross-Site Scripting (XSS).
- Cross-Site Request Forgery (CSRF).

- Robo del token mediante ataques XSS.
- Ataques de cambio de algoritmo

Nivel de seguridad

- Regenerar el ID de sesión tras el login.
- Implementar tiempos de inactividad cortos.
- Usar HTTPS exclusivamente.

- Configurar las banderas HttpOnly y Secure.
- Utilizar el atributo SameSite para mitigar CSRF.

- Usar firmas robustas (ej. RS256).
- Tiempos de expiración (exp) muy cortos.
- Implementar arquitectura de Refresh Tokens.